

UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA



Curso:

Base de Datos II

Laboratorio 05: Hash File

Alumno:

Llerena Silva, Nicolás Alejandro (202110190)

Profesor:

Lovon, Percy

Lima, setiembre de 2026

De qué va todo esto

Cuatro técnicas, un mismo problema: dónde meto un registro para poder encontrarlo después sin recorrer medio archivo. Las cuatro responden con la misma idea de fondo —una función que convierte la clave en una dirección— y se diferencian en algo que al principio parece un detalle y termina siendo lo importante: qué hacen cuando el bucket se llena.

La primera parte es código; las otras tres son ejercicios a mano sobre el mismo conjunto de 17 claves. Para el hash estático reutilicé el registro de 88 bytes y el `employee.csv` que vengo usando desde el laboratorio pasado (100 000 empleados sacados de la base del Laboratorio 01), así los números de I/O son comparables entre labs. Las trazas de P2, P3 y P4 las resolví primero en papel y después las verifiqué con un pequeño simulador, porque a la tercera vez que me equivoqué contando bits decidí que valía la pena. Los diagramas salen de ahí.

Un aviso sobre notación, para que no haya confusión más adelante. El enunciado define $\text{pos_bucket} = \text{Binary}(\text{hash}(\text{key}) \% 2^D)$ y, en P3, dice que se toman los d primeros dígitos del hash. Es decir: el hash es una cadena de D bits y el directorio se indexa por prefijos, leyendo de izquierda a derecha. Lo aclaro porque muchos textos usan los bits menos significativos y el resultado sale distinto; acá seguí al pie de la letra lo que pide el enunciado.

P1. Static Hashing

El archivo tiene M buckets primarios y ese número no se mueve nunca. Esa es toda la gracia y también toda la desgracia del método. La dirección de un registro sale de $h(k) = k \bmod M$ y, como M es constante, la dirección es estable: no hay directorio que consultar, no hay nivel que bajar, se calcula y se salta. Una lectura y listo. Cuando funciona, es la estructura más rápida de todas las que hemos visto en el curso.

La estructura del bucket

Cada bucket es un bloque del archivo y tiene esta forma:

```
+-----+-----+----- ... -----+  
| n_registros(4) | siguiente (4) | FB registros de 88 bytes |
```

```

+-----+-----+-----+ ... +-----+
cabecera de 8 bytes          cuerpo del bucket

n_registros : cuántos slots están ocupados
siguiente    : bloque del próximo eslabón de la cadena, o -1

```

El campo siguiente es el que arma el desbordamiento encadenado. Un bucket primario y sus buckets de overflow son, vistos desde afuera, una lista enlazada que empieza en una dirección calculada y sigue por punteros. Los overflow viven al final del archivo, después de los M primarios, y no tienen dirección propia: solo se llega a ellos siguiendo la cadena.

La cabecera del archivo guarda M, el factor de bloque FB, cuántos bloques hay en total y la cabeza de una lista de bloques libres. Esa última pieza es la que responde a la cuarta pregunta del enunciado, pero ya llegamos ahí.

Las cuatro operaciones

Inserción. Se calcula $h(k)$, se recorre la cadena y el registro entra en el primer slot libre que aparezca. Si la cadena entera está llena, se pide un bloque —de la lista de libres si hay alguno, si no uno nuevo al final del archivo— y se engancha como último eslabón. Costo: tantas lecturas como bloques tenga la cadena, más una escritura.

Búsqueda. Igual, pero sin escribir: se recorre la cadena comparando claves. Acá hay un detalle que la gente suele pasar por alto y que se ve clarísimo en las mediciones: la búsqueda que *<i>falla</i>* es más cara que la que acierta, porque obliga a recorrer la cadena completa antes de poder decir que no está. La que acierta se detiene cuando encuentra.

Eliminación. Se ubica el registro y se quita. Pero quitar y ya sería dejar el archivo lleno de huecos, así que el hueco se tapa con el último registro de la cadena. Es un truco barato —una lectura extra— y mantiene los buckets compactos hacia adelante, que es lo que hace que las búsquedas no se degraden con el uso.

¿Y si un bucket queda vacío?

Esta es la pregunta del enunciado y me parece que tiene dos respuestas, según de qué bucket estemos hablando.

Si el que se vacía es un **bucket de overflow**, hay que desengancharlo. El eslabón anterior pasa a apuntar a lo que apuntaba él, y su bloque se encadena en una lista de libres cuya cabeza vive en la cabecera del archivo. La próxima vez que haga falta un overflow, se toma de ahí en vez de crecer el archivo. Sin esto el archivo solo puede engordar, aunque se borre todo. En la prueba de estrés que aparece abajo, con $M = 7$ y 1800 bajas, quedaron 600 de 669 bloques reciclables: sin lista de libres, esos 600 bloques serían basura permanente ocupando disco.

Si el que se vacía es un **bucket primario**, no se toca. Y no es por pereza: su dirección es exactamente lo que devuelve $h(k)$, así que si lo elimináramos, la función hash apuntaría a un bloque que no existe. Un primario vacío es un costo que se paga por diseño; forma parte del espacio de direcciones. Lo más que se puede hacer es dejarlo marcado como vacío para que una búsqueda fallida termine en una sola lectura.

```
bash - labs/5

$ python3 static_hash.py
Static Hash File sobre data/employee.csv
buckets primarios : 101 ( $h(k) = k \bmod 101$ )
factor de bloque  : 8 registros por bucket
tamano de bloque  : 712 bytes = 8 de cabecera + 8 x 88
registros         : 1200 (factor de carga 1.49)
cadenas           : promedio 1.97 bloques, maxima 2, 98 buckets con overflow
search(10748) -> Yuchang Francisci, Human Resources [1 lecturas, bucket 42]
search(1) -> None [2 lecturas]
insert(999999) -> bucket 99, cadena [99, 164] [4L 1E]
bucket 99 antes  : cadena de 2 bloques [99, 164]
tras 8 bajas     : cadena de 1 bloques [99], lista de libres = 164
el bloque liberado se reutiliza en la siguiente alta: bloque 164
registros validos : 1193 (sin duplicados: 1193)
prueba de estres de la eliminacion:
M= 7 FB=3: 2000 altas y 1800 bajas -> quedan 200 registros, cadena maxima 12, 600 bloques reciclables
M= 23 FB=4: 2000 altas y 1800 bajas -> quedan 200 registros, cadena maxima 4, 454 bloques reciclables
M=101 FB=8: 2000 altas y 1800 bajas -> quedan 200 registros, cadena maxima 1, 195 bloques reciclables
$
```

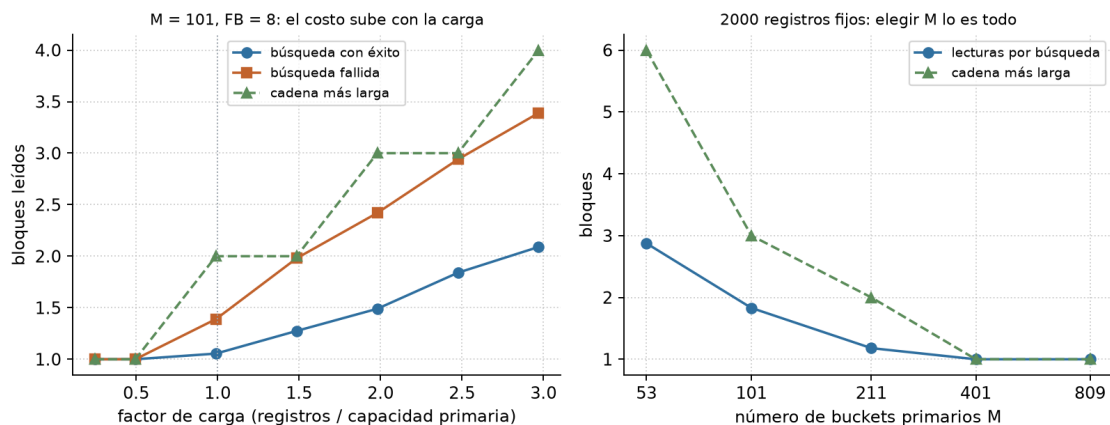
Las cuatro operaciones sobre 1200 empleados y la prueba de estrés con tres configuraciones. Fíjese en la línea de las bajas: la cadena pasa de 2 bloques a 1 y el bloque 164 queda reciclable.

Cuánto duele llenarlo

Medí dos cosas, porque el análisis de ventajas y desventajas sin números es pura opinión. La primera: qué pasa al subir el factor de carga con M fijo. Con carga 0,25 cada búsqueda cuesta una lectura y no hay una sola cadena; al llegar a 2,97 —o sea, con casi tres veces más registros que capacidad primaria— la búsqueda con éxito ya cuesta 2,09 lecturas, la fallida 3,39, y la

cadena más larga llega a 4 bloques. El hash dejó de ser $O(1)$ hace rato; ahora es una búsqueda lineal disfrazada.

La segunda: qué pasa si uno elige mal M . Con los mismos 2000 registros, $M = 53$ da cadenas de hasta 6 bloques y 2,88 lecturas por búsqueda, mientras que $M = 809$ deja todo en 1,00. Mismo algoritmo, mismos datos, mismo código: la diferencia es un número que se decidió al crear el archivo y que después no se puede cambiar sin reconstruirlo entero. Ahí está el problema del hashing estático, resumido en una gráfica.



Izquierda: el costo crece con la carga. Derecha: el mismo archivo con distinto M.

Un detalle honesto: los Employee_ID de este dataset son consecutivos, así que $k \bmod M$ los reparte casi perfecto. Es el mejor caso posible para la función módulo y las cadenas quedan parejas. Con claves agrupadas —fechas, códigos con prefijo, cualquier cosa que tenga estructura— la distribución se desbalancea y aparecen cadenas largas mucho antes de llegar a factor de carga 1. En ese escenario habría que cambiar la función hash por una que mezcle bits, tipo multiplicativa o un CRC.

P2. Extendible Hashing

Las claves son 10, 13, 34, 6, 23, 12, 15, 73, 28, 19, 67, 17, 41, 87, 57, 27 y 11, en ese orden. $FB = 3$, la profundidad global arranca en $D = 2$ y el archivo nace con dos buckets. Las reglas del enunciado, traducidas a algo que se pueda ejecutar sin dudar en cada paso, quedan así:

1. $h(k) = \text{Binary}(k \bmod 2^D)$ → cadena de D bits
2. el índice se recorre por prefijos: la entrada que es prefijo de $h(k)$
3. bucket con espacio → entra
4. bucket lleno y $|entrada| < D$ → SPLIT: la entrada se parte en dos

```

                                (entrada+'0' y entrada+'1') y se
redistribuye
5. bucket lleno y |entrada| = D  -> se encadena UN bucket de overflow (K = 1)
6. overflow lleno                -> REHASHING: D = D + 1 y se rehace el archivo

```

El punto 6 es el que hace que esta variante sea distinta del extendible hashing de libro. Acá la profundidad global no crece cuando un bucket se parte: crece solo cuando ya no queda más remedio, después de agotar el encadenamiento. Y ojo con una consecuencia que a mí me tomó un rato ver: como el hash es $k \bmod 2^D$, subir D **cambia el hash de todas las claves**. No es que se agregue un bit a la derecha y todo quede donde estaba; el 19, por ejemplo, pasa de 11 a 011, y su prefijo de dos bits deja de ser 11 para ser 01. Por eso el enunciado le llama rehashing y no simplemente duplicar el directorio: hay que volver a insertar todo desde cero.

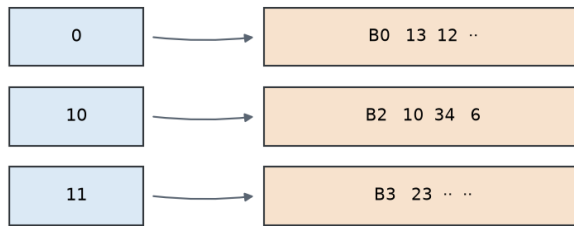
a) Inserción paso a paso

Primera fase, con $D = 2$. Los hashes son $k \bmod 4$ en dos bits.

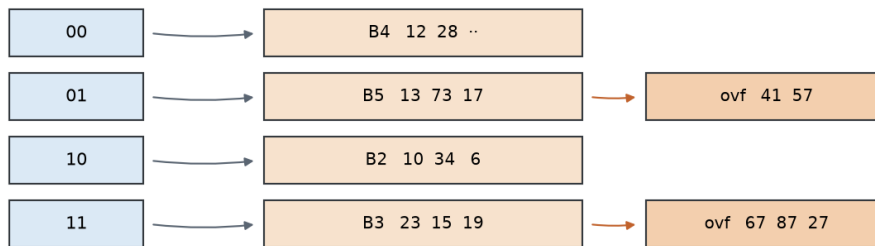
Clave	hash	Entrada del índice	Qué pasa
10	10	1	entra directo
13	01	0	entra directo
34	10	1	entra directo
6	10	1	entra directo
23	11	1	split en 10 / 11
23	11	11	entra directo
12	00	0	entra directo
15	11	11	entra directo
73	01	0	entra directo
28	00	0	split en 00 / 01
28	00	00	entra directo
19	11	11	entra directo
67	11	11	abre bucket de overflow
17	01	01	entra directo
41	01	01	abre bucket de overflow
87	11	11	va a la cadena
57	01	01	va a la cadena
27	11	11	va a la cadena
11	11	11	cadena llena: rehashing a $D=3$

Fase $D = 2$. Cuando una clave provoca un split aparece dos veces: la fila del split y la fila donde por fin entra.

a) $D = 2$, tras el split que provoca la clave 23



b) $D = 2$, cadenas llenas justo antes del rehashing



Dos momentos de la fase $D = 2$: arriba el primer split, abajo el estado justo antes del colapso.

Miren el segundo panel. Las entradas 01 y 11 llegaron a profundidad 2, que es la global, así que ya no pueden partirse: lo único que les queda es encadenar. Y encadenaron. Cuando llega el 11 —hash 11, la entrada más castigada— el bucket principal está lleno, el overflow también está lleno, y $K = 1$ prohíbe un segundo eslabón. Ahí se acabó: rehashing, $D = 3$, todo de nuevo.

Vale la pena notar qué tan desbalanceado estaba el archivo antes de reventar. Seis claves apiñadas en la entrada 11, otras cinco en la 01, y la entrada 10 con sus tres desde el principio sin haberse movido nunca. Con dos bits no había forma de separarlas.

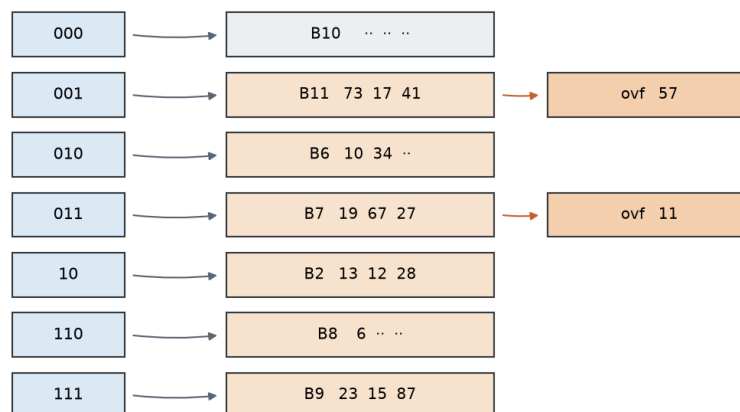
Segunda fase, $D = 3$. Ahora los hashes son $k \bmod 8$ en tres bits y el archivo vuelve a empezar con dos buckets.

Clave	hash	Entrada del índice	Qué pasa
10	010	0	entra directo
13	101	1	entra directo
34	010	0	entra directo
6	110	1	entra directo
23	111	1	entra directo
12	100	1	split en 10 / 11
12	100	10	entra directo
15	111	11	entra directo
73	001	0	entra directo

28	100	10	entra directo
19	011	0	split en 00 / 01
19	011	01	entra directo
67	011	01	split en 010 / 011
67	011	011	entra directo
17	001	00	entra directo
41	001	00	entra directo
87	111	11	split en 110 / 111
87	111	111	entra directo
57	001	00	split en 000 / 001
57	001	001	abre bucket de overflow
27	011	011	entra directo
11	011	011	abre bucket de overflow

Fase D = 3, la definitiva: las 17 claves entran sin volver a romper la cadena.

Estado final tras el rehashing (D = 3)



Estado final. Siete entradas de índice, dos cadenas de overflow y un bucket vacío.

Tres cosas que me parecen dignas de comentar del resultado final. La primera: la entrada 10 se quedó en profundidad 2 mientras sus vecinas llegaron a 3. Eso es extendible hashing haciendo bien su trabajo —solo se parte donde hace falta— y es exactamente lo que un directorio de tamaño fijo no puede hacer.

La segunda: el split de la entrada 00 produjo un bucket vacío (el 000) y otro que igual se desbordó (el 001, con 73, 17, 41 y el 57 colgando en overflow). Es que 73, 17, 41 y 57 tienen todos hash 001; ningún split los va a separar mientras D = 3. Partir sirve cuando las claves difieren en el bit siguiente. Si no difieren, uno parte, gasta un bucket y sigue igual de lleno. Esa

es la debilidad estructural del método y por eso hace falta el encadenamiento como red de seguridad.

La tercera: quedaron 17 claves repartidas en 7 buckets con capacidad 3, o sea 21 slots primarios más 6 de overflow. La ocupación anda por el 63 %, que suena bajo hasta que uno recuerda que el precio de tener búsquedas de una lectura es justamente dejar aire en los buckets.

b) Algoritmo de búsqueda

Con esta variante hay tres casos: la clave está en el bucket principal, está en el overflow, o no está. El algoritmo tiene que cubrir los tres y —esto es lo importante en memoria secundaria— tiene que hacerlo con el mínimo de lecturas.

```
buscar(k):  
  h <- Binary(k mod 2^D)          # D bits, el D actual del archivo  
  e <- entrada del índice que es prefijo de h  
  si no existe e: return NO_ESTA   # índice sin cobertura, no debería pasar  
  
  b <- leer_bucket(puntero(e))     # 1ra lectura a disco  
  si k está en b: return registro  
  
  mientras b.overflow != NULL:     # a lo más K = 1 vueltas  
    b <- leer_bucket(b.overflow)   # 2da lectura  
    si k está en b: return registro  
  
  return NO_ESTA
```

Costo: 1 lectura en el caso bueno, 1 + K en el peor. Con K = 1 eso son dos lecturas y ni una más, sin importar cuántos registros tenga el archivo. El directorio se asume en RAM —son pares (prefijo, puntero), unos pocos KB incluso para archivos grandes—; si no cupiera, habría que sumarle una lectura al índice.

c) Algoritmo de eliminación

Eliminar es buscar y borrar, sí, pero lo interesante viene después: qué hacer con el espacio que se libera.

```
eliminar(k):  
  e <- entrada del índice que es prefijo de Binary(k mod 2^D)  
  b <- leer_bucket(puntero(e))  
  ubicar k en b o en su cadena de overflow; si no está: return FALSE  
  
  quitar k del bucket donde estaba  
  
  # 1. compactar la cadena: el hueco se tapa con el último registro  
  si el hueco quedó en el principal y el overflow tiene registros:
```

```

    mover un registro del overflow al principal

# 2. soltar el overflow si se vació
si el overflow quedó vacío:
    principal.overflow <- NULL
    encolar ese bloque en la lista de libres

# 3. fusionar entradas hermanas si el contenido cabe en una
hermana <- entrada que difiere de e solo en el último bit
si hermana existe, es hoja, y |b| + |hermana| <= FB:
    volcar ambos buckets en uno
    reemplazar e y hermana por su prefijo común en el índice
    liberar el bucket sobrante
    repetir el paso 3 con el prefijo común (la fusión se propaga)

return TRUE

```

Sobre la gestión de buckets libres, que es lo que pregunta el enunciado: los bloques liberados no se devuelven al sistema operativo ni se recorta el archivo. Se encadenan en una lista de libres cuya cabeza está en la cabecera del archivo, usando el propio campo de puntero del bucket vacío para enlazar al siguiente libre. Cuesta cero espacio extra y convierte cada bloque muerto en un bloque reciclable. Es exactamente lo que implementé en P1 y funciona igual acá.

El paso 3, la fusión, admite discusión. Fusionar mantiene el índice chico y la ocupación alta, pero si el archivo tiene altas y bajas alternadas alrededor del umbral uno termina partiendo y fusionando el mismo par de buckets una y otra vez —el clásico thrashing— y eso son dos escrituras cada vez. Yo dejaría la fusión detrás de una histéresis: fusionar solo cuando la suma baje de algo así como $FB/2$, no cuando simplemente quepa. O directamente no fusionar y reconstruir el archivo cada tanto, que es lo que hacen varios motores reales.

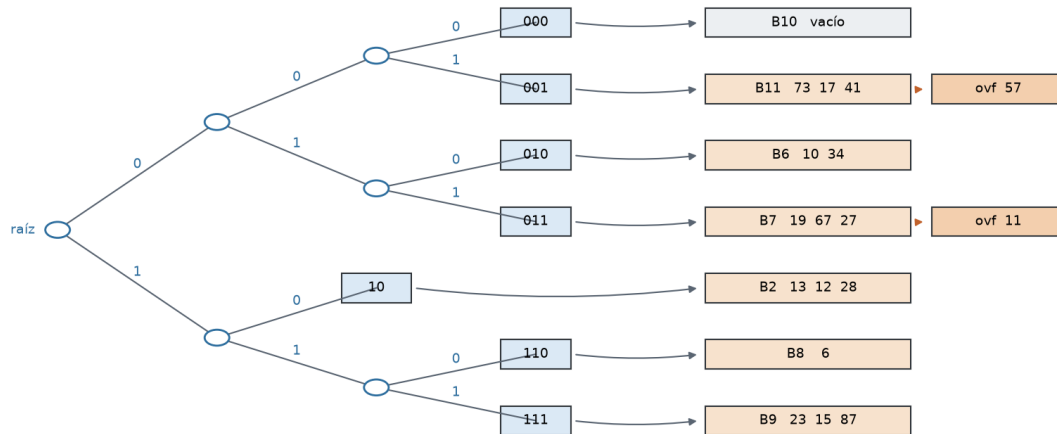
P3. Trie Hashing

Mismo problema, mismas claves, pero ahora el directorio no es una lista de prefijos sino un árbol binario. Cada arista es un bit del hash, se baja consumiendo bits de izquierda a derecha, y las hojas son las que guardan el puntero al bucket. La profundidad global es $D = 3$ desde el inicio, así que los hashes son $k \bmod 8$ en tres bits y el árbol arranca con dos hojas, 0 y 1.

Acá viene lo que más me gustó de resolver los dos ejercicios seguidos: el reparto final es **idéntico** al de la segunda fase de P2. Y tiene que serlo. Las dos técnicas usan la misma función hash con la misma D y la misma regla de

partir por el bit siguiente; lo único que cambia es cómo se guarda el mapa. En P2 el mapa es una tabla de prefijos; acá es un trie donde cada prefijo es un camino desde la raíz. Distinta estructura de datos, misma partición.

Trie de directorio con $D = 3$: las hojas guardan el puntero al bucket



El árbol final. Las siete hojas coinciden una a una con las siete entradas del índice de P2.

Las hojas quedaron en 000, 001, 010, 011, 10, 110, 111. La rama de la izquierda bajó hasta el tercer nivel porque las claves con hash 001 y 011 se apiñaron; la rama 10, en cambio, se quedó en el segundo nivel y nunca necesitó hijos. Un árbol deliberadamente desbalanceado, y está bien que lo esté: se profundiza donde hay densidad de claves y no donde no la hay.

Algoritmo de inserción

```

insertar(k, registro):
  h <- Binary(k mod 2^D)
  nodo <- raíz; d <- 0
  mientras nodo no sea hoja:      # bajar consumiendo bits
    nodo <- hijo[ h[d] ]; d <- d + 1

  b <- leer_bucket(nodo.puntero)
  si b tiene espacio: escribir y return

  si d < D:                        # todavía se puede profundizar
    convertir la hoja en nodo interno
    crear dos hojas nuevas: h||0 y h||1, con un bucket cada una
    repartir los registros de b según su bit d
    reintentar la inserción de k      # puede volver a partirse
  sino:                            # d = D, no hay más bits que mirar
    si b.overflow == NULL y K > 0:
      enganchar un bucket de overflow con k
    sino:
      REHASHING con D = D + 1
  
```

Lo de reintentar en vez de insertar directo importa: cuando las cuatro claves comparten el bit d , la partición deja todo de un lado y el bucket sigue lleno. Reintentando, el algoritmo vuelve a evaluar y parte otra vez si aún puede. Es lo que pasó con la entrada 00 de P2.

Algoritmo de búsqueda

```
buscar(k):
  h <- Binary(k mod 2^D)
  nodo <- raiz; d <- 0
  mientras nodo no sea hoja:
    nodo <- hijo[ h[d] ]; d <- d + 1
    si nodo == NULL: return NO_ESTA      # rama inexistente

  b <- leer_bucket(nodo.puntero)        # única lectura a disco
  si k está en b: return registro
  si b.overflow != NULL:
    return buscar_en(b.overflow, k)
  return NO_ESTA
```

El descenso por el trie no cuesta I/O: el árbol vive en RAM, son punteros. Lo único que toca disco es leer el bucket de la hoja. Por eso el costo es el mismo que en P2 —una lectura, dos si hay que mirar el overflow— aunque el árbol tenga cinco niveles.

Algoritmo de eliminación (opcional)

```
eliminar(k):
  localizar la hoja como en buscar(k)
  si k no está: return FALSE
  quitar k; compactar la cadena; liberar el overflow si quedó vacío

  # colapso del árbol, de abajo hacia arriba
  p <- padre de la hoja
  mientras p tenga sus dos hijos como hojas
    y |hijo_izq| + |hijo_der| <= FB:
      volcar los dos buckets en uno
      p pasa a ser hoja apuntando a ese bucket
      liberar el bucket sobrante
      p <- padre de p
```

El colapso es la operación inversa de la partición y solo se puede aplicar cuando los dos hijos son hojas: si uno es interno, sus descendientes viven más abajo y fusionar significaría rehacer todo un subárbol. Igual que en P2, yo lo pondría detrás de un umbral con histéresis para no entrar en un ciclo de partir y colapsar.

P4. Linear Hashing

Litwin propuso esto en 1980 con una idea que, la primera vez que uno la lee, suena a trampa: crecer el archivo **sin directorio** y sin que el bucket que se parte tenga nada que ver con el bucket que se desbordó. Suena mal. Funciona sorprendentemente bien.

Los ingredientes son tres: el nivel L , que dice cuántas veces se duplicó el archivo; el puntero p , que marca cuál es el próximo bucket en la fila para partirse; y dos funciones hash encadenadas, $h_L(k) = k \bmod (N \cdot 2^L)$ y su versión con el doble de rango. Con $N = 2$ buckets iniciales y $FB = 3$, el direccionamiento queda así:

```
dirección(k):  
  i <- k mod (N * 2^L)  
  si i < p:                               # ese bucket ya se partió en esta ronda  
    i <- k mod (N * 2^(L+1))  
  return i
```

La política que usé para disparar el split es la clásica: **cada vez que hay un desbordamiento** se parte el bucket p , sea cual sea. También se puede disparar por factor de carga, y en un sistema real eso da mejor control de la ocupación, pero para 17 claves la regla del overflow es la que deja la traza más legible.

1. Paso a paso con los datos de P2

Las 17 claves de P2 entran en ese orden, con $N = 2$ buckets iniciales y $FB = 3$. El archivo arranca con B0 y B1 y termina con nueve buckets. Cada recuadro de la figura es el estado después de un split; el bucket sombreado es al que apunta p , o sea el próximo en partirse.

inicio: 2 buckets · L=0, p=0

B0	—
B1	—

12: overflow B0 · split B0→B2 · L=0, p=1

B0	12
B1	13 23
B2	10 34 6

p

p

73: overflow B1 · split B1→B3 · L=1, p=0

B0	12
B1	13 73
B2	10 34 6
B3	23 15

p

67: overflow B3 · split B0→B4 · L=1, p=1

B0	—
B1	13 73
B2	10 34 6
B3	23 15 19
B4	12 28

p

67

Primeros cuatro estados: el archivo pasa de dos a cinco buckets.

41: overflow B1 · split B1→B5 · L=1, p=2

B0	—	
B1	73 17 41	
B2	10 34 6	
B3	23 15 19	▶ 67
B4	12 28	
B5	13	

87: overflow B3 · split B2→B6 · L=1, p=3

B0	—	
B1	73 17 41	
B2	10 34	
B3	23 15 19	▶ 67 87
B4	12 28	
B5	13	
B6	6	

57: overflow B1 · split B3→B7 · L=2, p=0

B0	—	
B1	73 17 41	▶ 57
B2	10 34	
B3	19 67	
B4	12 28	
B5	13	
B6	6	
B7	23 15 87	

11: overflow B3 · split B0→B8 · L=2, p=1

B0	—	
B1	73 17 41	▶ 57
B2	10 34	
B3	19 67 27	▶ 11
B4	12 28	
B5	13	
B6	6	
B7	23 15 87	
B8	—	

Los cuatro restantes, hasta el estado final de nueve buckets.

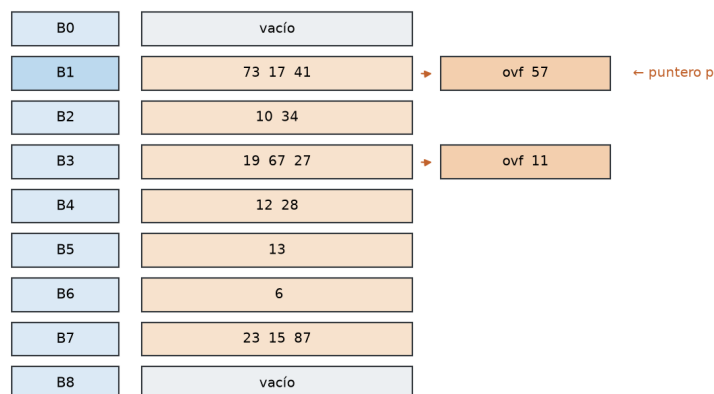
Y la misma secuencia en tabla, clave por clave:

Clave	Bucket	L	p	Qué pasa
10	B0	0	0	entra directo
13	B1	0	0	entra directo
34	B0	0	0	entra directo
6	B0	0	0	entra directo
23	B1	0	0	entra directo
12	B0	0	1	overflow en B0, split de B0 en B2

15	B1	0	1	entra directo
73	B1	1	0	overflow en B1, split de B1 en B3, sube a nivel 1
28	B0	1	0	entra directo
19	B3	1	0	entra directo
67	B3	1	1	overflow en B3, split de B0 en B4
17	B1	1	1	entra directo
41	B1	1	2	overflow en B1, split de B1 en B5
87	B3	1	3	overflow en B3, split de B2 en B6
57	B1	2	0	overflow en B1, split de B3 en B7, sube a nivel 2
27	B3	2	0	entra directo
11	B3	2	1	overflow en B3, split de B0 en B8

Los valores de L y p son los que quedan después de procesar la clave.

Linear hashing: estado final con nivel = 2, p = 1 y 9 buckets



Estado final: nueve buckets, dos cadenas cortas y el puntero listo para la próxima ronda.

Hay un momento en la traza que vale su peso en oro para entender el método. La última clave, el 11, se desborda en B3 y el algoritmo, obediente, parte... B0. Un bucket vacío. Lo parte porque le tocaba el turno, no porque hiciera falta. Resultado: dos buckets vacíos donde había uno, y el desbordamiento de B3 sigue ahí, colgando de su cadena.

Suena absurdo dicho así, pero es el precio deliberado del diseño. Al partir siempre en orden, la dirección de un bucket se puede calcular con dos módulos y una comparación —sin leer ningún directorio, sin ninguna estructura auxiliar en RAM— y el archivo crece de a un bucket en vez de duplicarse de golpe.

Amortizado, el puntero termina pasando por todos y el desbalance se corrige solo. A cambio, en el corto plazo uno convive con cadenas que podrían haberse evitado.

Puesto al lado del extendible de P2, el contraste es directo. Los dos terminan agrupando por los bits bajos de la clave —B1 junta 73, 17, 41 y 57, que son justamente los que dejan residuo 1 al dividir entre 8; B7 junta 23, 15 y 87, los de residuo 7— pero llegan ahí por caminos distintos. El extendible parte el bucket que se desbordó, en el momento en que se desborda. El lineal parte el que le toca por turno, y por eso acabó separando 12 y 28 de 13 en dos buckets (B4 y B5) sin que nadie se lo pidiera, y partiendo B0 estando vacío. Extendible parte por necesidad; lineal parte por disciplina.

2. Algoritmo de inserción

```
insertar(k, registro):
  i <- dirección(k)
  b <- leer_bucket(i)
  si b tiene espacio:
    escribir registro en b
  sino:
    enganchar/usar bucket de overflow de la cadena de i

    # el split va aparte, y siempre sobre p
    base <- N * 2^L
    nuevo <- p + base
    repartir los registros de la cadena de p entre p y nuevo
      usando k mod (2 * base)
    p <- p + 1
    si p == base:
      # se completó la ronda
      p <- 0; L <- L + 1
```

3. Algoritmo de búsqueda

```
buscar(k):
  i <- k mod (N * 2^L)
  si i < p: i <- k mod (N * 2^(L+1))

  b <- leer_bucket(i)
  mientras b != NULL:
    si k está en b: return registro
    b <- leer_bucket(b.overflow)
  return NO_ESTA
```

Dos operaciones aritméticas y una lectura. Sin índice que consultar, sin nivel que bajar, sin nada en RAM más que dos enteros: L y p. Para mí esa es la mejor propiedad del linear hashing y la razón de que se use en tablas hash en memoria y en algunos motores embebidos.

Ventajas y desventajas: las cuatro, cara a cara

Después de implementar una y resolver tres a mano, la comparación se me ordenó sola alrededor de una pregunta: ¿qué hace cada técnica cuando el bucket se llena? Ahí está toda la diferencia.

Técnica	Ventajas	Desventajas	Cuándo la usaría
Static Hashing	Una lectura por búsqueda mientras la carga sea baja. Nada de directorio ni de estructuras auxiliares. Es el más simple de implementar y de recuperar ante fallos.	M se fija al crear el archivo y no se puede cambiar sin reconstruir todo. Las cadenas degradan la búsqueda a lineal. Un primario vacío no se puede eliminar nunca.	Archivos de tamaño conocido y estable: catálogos, tablas de referencia, un diccionario que se carga una vez.
Extendible Hashing	Dos lecturas como techo (una si el directorio está en RAM y no hay overflow). Crece solo donde hay densidad de claves; los buckets que no se llenan no se tocan.	El directorio puede duplicarse de golpe y ocupar bastante RAM. Con esta variante, el rehashing obliga a releer y reescribir el archivo entero. Si las claves colisionan en los D bits, partir no separa nada.	Datos con crecimiento impredecible y distribución despereja, que es la mayoría de los casos reales.
Trie Hashing	El directorio es explícito y navegable: se ve dónde se profundizó y dónde no. El descenso es en RAM, así que el I/O es el mismo que el de extendible.	Más punteros y más nodos internos que una tabla plana de prefijos. Con claves muy agrupadas el árbol se desbalancea y el camino se alarga (aunque no cueste I/O).	Cuando el directorio no cabe cómodo en memoria y conviene paginarlo por subárboles, o cuando hace falta inspeccionar la estructura.
Linear Hashing	Cero directorio: la dirección sale de dos módulos y una comparación. El archivo crece de a un bucket, sin pausas ni duplicaciones bruscas. Ocupación pareja en el largo plazo.	El bucket que se parte no es el que se desbordó, así que uno convive con cadenas evitables. Puede partir buckets vacíos. Necesita una política de disparo bien elegida.	Archivos que crecen de forma sostenida y donde no se quiere pagar RAM por un directorio, o donde el crecimiento debe ser gradual y predecible.

Las cuatro técnicas resuelven la búsqueda por igualdad en un puñado de lecturas. Ninguna sirve para rangos: eso es territorio del B+.

Si tuviera que quedarme con una sola frase por técnica: el estático apuesta todo a que uno adivine bien M; el extendible paga un directorio a cambio de partir solo donde duele; el trie es el mismo extendible con el directorio hecho árbol; y el lineal renuncia al directorio a cambio de aceptar cadenas que sabe que podría haber evitado.

Una observación transversal que me quedó dando vueltas: las tres técnicas dinámicas terminaron con particiones casi iguales sobre las mismas 17 claves.

No es casualidad ni suerte. Con la misma función hash y la misma capacidad por bucket, el reparto final está prácticamente determinado; lo que cada técnica decide de verdad es *cuándo* crece, *cuánto* crece de una vez y *cuánta* memoria gasta para saber dónde está cada cosa. Tres decisiones de ingeniería sobre el mismo esqueleto matemático.

Y una limitación honesta de todo el laboratorio: 17 claves son pocas. Alcanzan para ver los mecanismos —splits, cadenas, rehashing, el puntero avanzando— pero no para hablar de rendimiento con seriedad. Por eso el único número duro de este informe sale de P1, que sí corre sobre miles de registros. Las otras tres se juzgan por su comportamiento estructural, no por milisegundos.

Lo que me llevo

Que el hashing no es una técnica sino una familia, y que lo que las separa no es la función hash —todas usan módulo— sino la política de crecimiento. El estático no crece. El extendible crece donde le duele. El lineal crece por turnos. El trie crece como el extendible pero se lo cuenta a uno mejor.

También que la pregunta del bucket vacío, que parecía un detalle menor del enunciado, es de las cosas más prácticas del lab: sin una lista de libres, un archivo hash con muchas bajas se convierte en un queso gruyere que solo sabe crecer. Lo medí en P1 y no es teórico: con $M = 7$ quedaron 600 de 669 bloques reciclables después de las bajas.

Si tuviera que seguir por algún lado, probaría la variante del extendible con duplicación de directorio en lugar de rehashing completo, para comparar cuánto I/O se ahorra evitando reinsertar las 17 claves. Intuyo que bastante, aunque a cambio de un directorio más grande. Queda para otra vez.

Anexo: archivos entregados

Con este informe va el código de P1, que es la única parte del laboratorio que pedía implementación. `static_hash.py` contiene la clase `StaticHashFile`: el bucket con su cabecera, las cuatro operaciones, el desbordamiento encadenado, la lista de bloques libres y las estadísticas de cadena; ejecutándolo directamente se reproducen la demostración y la prueba de estrés que aparecen más arriba. `registro.py` define el registro de 88 bytes y la lectura

del CSV —es el mismo módulo del laboratorio anterior— y los datos van en `data/employee.csv`. P2, P3 y P4 son ejercicios a mano y su solución completa está en este documento.